Chapter 3

Robot motion and task planning

An automated motion and task planning framework for a mobile robot is introduced in this chapter as the nominal solution. In particular, a finite-state transition system is constructed to serve as the discrete abstraction of the robot motion within the workspace. The robot is assigned a complex task which is specified as temporal logic formulas over this transition system. Through the model-checking algorithm, we synthesize firstly a discrete motion plan that satisfies the given task and more importantly minimizes a total cost. Then based on this discrete plan, we construct a hybrid control strategy that drives the robot such that its final trajectory satisfies the task specification.

3.1 Abstraction of robot motion

The robot's motion within a certain workspace is abstracted as a finite-state transition system (FTS) [11]. This FTS is constructed by integrating two aspects: (i) the workspace model; (ii) the robot dynamics and its navigation controller.

3.1.1 The workspace model

The workspace we consider is a bounded n-dimensional space, denoted by $\mathcal{W}_0 \subset \mathbb{R}^n$, within which there exists W smaller regions of interest $\pi_i \subset \mathcal{W}_0$, $\forall i = 1, 2, \cdots, W$. Denote by $\Pi = \{\pi_1, \cdots, \pi_W\}$ the set of all smaller regions. We require that Π is a full partition of the workspace and any two regions $\pi_i, \pi_j \in \Pi$ do not overlap. Namely, $\bigcup_{i=1}^{W} \pi_i = \mathcal{W}_0$ and $\pi_i \cap \pi_j = \varnothing$, $\forall i, j = 1, 2, \ldots, W$ and $i \neq j$.

Atomic propositions are boolean variables that can be either `True` or `False`, which are denoted by $\top$ and $\bot$ for brevity. They are used to express known properties about the state of the robot. Specifically, in order to indicate the robot's position, we define the set of atomic propositions $AP_r = \{a_{r,i}\}$, $i = 1, \cdots, W$, where

$$a_{r,i} = \begin{cases} \top & \text{if the robot is in region } \pi_i, \\ \bot & \text{otherwise,} \end{cases} \tag{3.1}$$

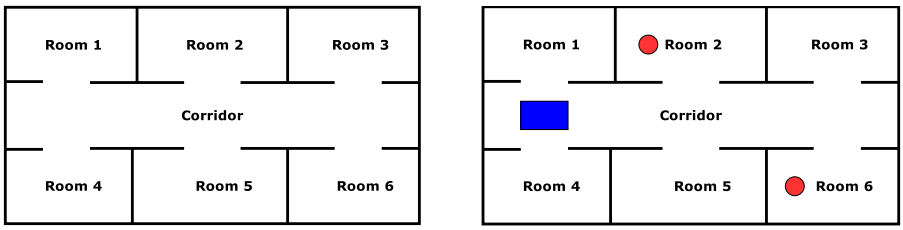

Figure 3.1: Left: the office-like partition of six rooms and one corridor. Right: after adding in AP_p, with two balls (in red) and one basket (in blue).

where $a_{r,i}$ can be evaluated by monitoring the measurements from a localization or positioning system. The requirement that $\bigcup_{i=1}^{n} \pi_i = \mathcal{W}_0$ is important to ensure that the robot's position is tracked at all time. Beside the geometric structure of the workspace, we also would like to express some generic properties within the workspace that are of interest to potential tasks. Denote by $AP_p = \{a_{p,1}, a_{p,2}, \cdots, a_{p,M}\}$ the set of atomic propositions for these properties. For simplicity, we set $AP = AP_r \cup AP_p$ as the set of all atomic propositions.

Definition 3.1. The labeling function $L : \Pi \to 2^{AP}$ maps a region $\pi_i \in \Pi$ to the set of atomic propositions satisfied by π_i, and $a_{r,i} \in L(\pi_i)$ by default, $\forall i = 1, \cdots, W$. ▲

Note that partial satisfaction of a proposition is not allowed. Namely, if only a part of region π_i satisfies $a \in AP$ and the other part does not, then π_i should be split into two regions: one satisfies a and the other does not. AP_p greatly improves the flexibility when expressing the intended tasks because one generic property can represent one type of regions without explicitly specify the location of these regions.

Example 3.1. An office-like workspace has six rooms and one corridor, which gives the partition in Figure 3.1. Two properties are "there is a basket in the region" and "there is a ball in the region". ▲

Additionally, since every region is a dense subset of the n-dimensional space, it is impossible to represent each region by the set of points contained in it. Thus it is crucial to represent and encode these regions efficiently. For instance, rectangles can be encoded by its center coordinate, height and width; sphere by its center and radius; triangular by the coordinates of its corners. There are also automated partition tools like Delaunay triangulation [100] and Voronoi diagram [97]. Note that this level of partition is preliminary and not robot-specific.

3.1.2 Robot dynamics and navigation controller

We assume the robot satisfies the following continuous dynamics:

$$\dot{x}(t) = f(x(t), u(t)), \tag{3.2}$$

where $x \in \mathbb{R}^n$, $u \in \mathbb{R}^m$ are the position and control signal; $f(\cdot)$ is Lipschitz continuous [82]. Thus the system is deterministic, i.e., given an initial state $x(0)$, a control input $u(t) : \mathbb{R} \to \mathbb{R}^m$ produces an unique state trajectory. We mainly take into account the single-integrator dynamics or the unicycle model. However the proposed framework can be potentially applied to other dynamics.

Given the preliminary partition from Section 3.1.1 and the agent dynamics by (3.2), we need to abstract the robot's ability to transit from one region to another, which is not necessarily the adjacency relation in the geometrical sense. Instead it is defined in the control-driven fashion.

Definition 3.2. There is a transition from π_i to π_j if an admissible navigation controller $\mathcal{U} : \mathbb{R}^n \times \Pi \times \Pi \to \mathbb{R}^m$:

$$u(t) = \mathcal{U}(x(t), \pi_i, \pi_j), \qquad t \in [t', t''] \tag{3.3}$$

exists that could drive the system (3.2) from **any** point in region π_i to a point in region π_j in finite time. At the same time the robot should stay within π_i or π_j. Namely, $x(t') \in \pi_i$, $x(t'') \in \pi_j$ and $x(t) \in \pi_i \cup \pi_j$, $\forall t \in [t', t'']$. ▲

The above definition is closely related to the implementation of a discrete motion plan described in Section 3.3.3. Two main navigation techniques are discussed in the thesis: (i) [103] proposes a potential-field-based feedback control algorithm. It navigates a differential-driven mobile robot from any point inside a region to an adjacent region through a desired facet. The partition is based on generalized Voronoi diagram and a smooth vector filed is constructed over each triangular region; (ii) [87] provides a provably correct point-to-point navigation algorithm, by constructing an exact navigation function for sphere workspace and obstacles. It has been successfully applied in both single [105] and multi-agent [24] navigation control under different geometric constraints, like single-integrator [106], double-integrator [23] vehicles. By following the negated gradient of the navigation function, a collision free path is guaranteed from almost any initial position in the free space to any goal position in the free space given that the workspace is valid.

It is rarely the case that a navigation technique is applicable to any type of partitions. For instance, the potential-filed-based method requires a triangular partition while the navigation-function-based approach needs all sphere structures. This means that the preliminary workspace partition might be modified in terms of number, size and shape of the regions. Example 3.2 shows some cases where the preliminary partition is modified after incorporating the robot dynamics and navigation techniques. This might lead to an over-approximation or under-approximation [129] of the actual workspace as some regions are shrunk or expanded during the process. In some cases, another navigation technique needs to be designed when it is infeasible to incorporate one navigation technique to a given workspace model.

Example 3.2. As shown in Figure 3.2, the office-like workspace from Figure 3.1 is further partitioned since the underlying navigation technique relies on triangular

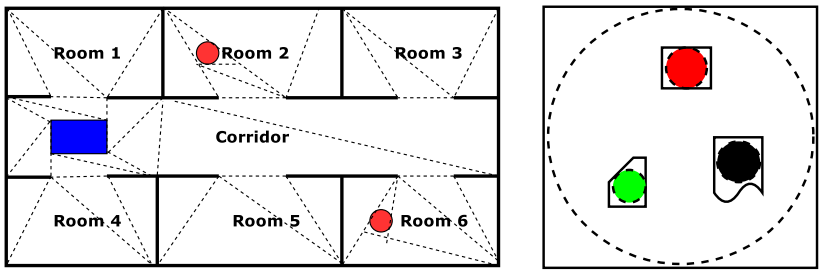

Figure 3.2: Results of workspace abstraction after incorporating the navigation techniques, as explained in Example 3.2.

partition. The irregular regions in the right image are approximated by circular areas due to its navigation-function-based construction. ▲

3.1.3 Control-driven and weighted FTS

The robot motion is abstracted as transitions among the regions $\Pi = \{\pi_1, \cdots, \pi_N\}$. With a slight abuse of notation, we denote by $\pi_i = \{\texttt{the robot is at region}\ \pi_i\}$, $i = 1, \cdots, N$, the states that reflect which region the robot is currently visiting. The transitions or state changes represent that the robot has moved from one region to another. Recall that the transition relation is not necessarily the adjacency relation in the geometrical sense, but by Definition 3.2. Formally the control-driven and weighted finite-state transition system (FTS) is defined below:

Definition 3.3. The weighted FTS is a tuple $\mathcal{T}_c = (\Pi, \longrightarrow_c, \Pi_0, AP, L_c, W_c)$, where $\Pi = \{\pi_i, i = 1, \cdots, W\}$ is the set of states; $\longrightarrow_c \subseteq \Pi \times \Pi$ is the transition relation by Definition 3.2. For simplicity, $\pi_i \longrightarrow_c \pi_j$ is equivalent to $(\pi_i, \pi_j) \in \longrightarrow_c$; $\Pi_0 \subseteq \Pi$ is the initial state, to indicate where the robot may start from; $AP = AP_r \cup AP_p$ is the set of atomic propositions; $L_c : \Pi \to 2^{AP}$ is a labeling function by Definition 3.1; $W_c : \Pi \times \Pi \to \mathbb{R}^+$ is the weight function as the cost of a transition in $\longrightarrow_c$. ▲

We assume that $\mathcal{T}_c$ does not have a terminal state. The successors of state π_i are defined as $\texttt{Post}(\pi_i) = \{\pi_j \in \Pi \,|\, \pi_j \longrightarrow_c \pi_j\}$. An infinite *path* of $\mathcal{T}_c$ is an infinite state sequence $\tau = \pi_1\pi_2\cdots$ such that $\pi_1 \in \Pi_0$ and $\pi_i \in \texttt{Post}(\pi_{i-1})$ for all $i > 0$. The *trace* of a path is the sequence of sets of atomic propositions that are true in the states along that path, i.e., $\texttt{trace}(\tau) = L_c(\pi_1)L_c(\pi_2)\cdots$. The trace of $\mathcal{T}_c$ is defined as $\texttt{Trace}(\mathcal{T}_c) = \cup_{\tau \in I}\, \texttt{trace}(\tau)$, where I is the set of all infinite paths in $\mathcal{T}_c$. An useful way to represent an infinite path is to use the ω-operator, to indicate the segment that has to be repeated infinitely many times [11].

The weighted FTS $\mathcal{T}_c$ is *fully-known* if it reflects the actual workspace model and robot dynamics; $\mathcal{T}_c$ is called *static* if $\mathcal{T}_c$ does not change with time.

3.2 Task specification as LTL formulas

A language is needed to specify a complex task, which on one hand should be expressive enough to specify various types of tasks, and on the other hand should be formal enough to avoid ambiguity and misinterpretation. Linear time Temporal Logic (LTL) provides a concise and formal way to specify both propositional and temporal constraints on the system behavior.

3.2.1 Syntax and semantics

Linear-time temporal logic (LTL) is defined using the following syntax:

$$\varphi ::= \top \mid a \mid \varphi_1 \vee \varphi_2 \mid \neg\varphi \mid \bigcirc\varphi \mid \varphi_1 \,\mathsf{U}\, \varphi_2, \tag{3.4}$$

where $a \in AP$ and $\wedge$ (*or*), $\neg$ (*not*), $\bigcirc$ (*next*), U (*until*). For brevity, we omit the derivations of other useful operators like $\Box$ (*always*), $\Diamond$ (*eventually*), $\Rightarrow$ (*implication*) and refer to Chapter 5 of [11]. An infinite *word* over the alphabet 2^{AP} is an infinite sequence $\sigma \in (2^{AP})^\omega$ that $\sigma = S_0\, S_1\, S_2 \cdots$, where $S_k \in 2^{AP}$ for all $k = 1, 2, \cdots$, where S_k is the set of atomic propositions that are true at time step k.

The semantics of LTL for an infinite word σ is given via a doubly-recursive definition of the relation $(\sigma, k) \vDash \varphi$, i.e., σ *satisfies* φ at time step k.

Definition 3.4. The semantics of LTL is defined as follows:

$$\begin{aligned}
(\sigma, k) \vDash a \quad &\leftrightarrow \quad a \in S_k \\
(\sigma, k) \vDash \neg\varphi \quad &\leftrightarrow \quad (\sigma, k) \nvDash \varphi \\
(\sigma, k) \vDash \bigcirc\varphi \quad &\leftrightarrow \quad (\sigma, k+1) \vDash \varphi \\
(\sigma, k) \vDash \varphi_1 \vee \varphi_2 \quad &\leftrightarrow \quad (\sigma, k) \vDash \varphi_1 \text{ or } (\sigma, k) \vDash \varphi_2 \\
(\sigma, k) \vDash \varphi_1 \,\mathsf{U}\, \varphi_2 \quad &\leftrightarrow \quad \exists k' \in [k, +\infty],\ (\sigma, k') \vDash \varphi_2 \text{ and} \\
&\qquad\quad \forall k'' \in (k, k'),\ (\sigma, k'') \vDash \varphi_1 .
\end{aligned}$$

▲

Any LTL formula φ is satisfied by σ at time step 0 if $(\sigma, 0) \vDash \varphi$ (for simplicity we denote by $\sigma \vDash \varphi$). The *words* of φ is defined as the set of words that satisfy φ at time step 0, i.e., $\texttt{Words}(\varphi) = \{\sigma \in (2^{AP})^\omega \mid \sigma \vDash \varphi\}$. Given an infinite path τ of $\mathcal{T}_c$ and an LTL formula φ over AP, $\texttt{trace}(\tau)$ is a word over the alphabet 2^{AP}. Thus we can verify if $\texttt{trace}(\tau)$ satisfies φ according to the semantics (3.4).

Definition 3.5. An infinite path τ **satisfies** φ, i.e., $\tau \vDash \varphi$ if its trace $\texttt{trace}(\tau) \vDash \varphi$. A satisfying path is also called a **plan** for φ. ▲

LTL formulas can be used to specify various robot control tasks, such as safety ($\Box\neg\varphi_1$, globally avoiding φ_1), ordering ($\Diamond(\varphi_1 \wedge \Diamond(\varphi_2 \wedge \Diamond\varphi_3))$, φ_1, φ_2, φ_3 hold in sequence), response ($\varphi_1 \Rightarrow \varphi_2$, if φ_1 holds, φ_2 will hold in future), repetitive surveillance ($\Box\Diamond\varphi$, φ holds infinitely often).

Another particular class of LTL we consider in this thesis is the syntactically co-safe LTL (sc-LTL) [93]. They only contain the $\bigcirc$, U and $\Diamond$ operators and are

written in positive normal form [35]. In contrast, the satisfaction of an sc-LTL formula can be achieved in a finite time, i.e., each word satisfying an sc-LTL formula φ consists of a *satisfying prefix* that can be followed by an arbitrary suffix.

3.2.2 Problem formulation

As discussed in the introduction, a single counter example would be enough for the purpose of verification, i.e., to verify that not all infinite paths of $\mathcal{T}_c$ satisfy φ. However for plan synthesis, since the derived plan needs to be implemented by autonomous robots, we need to find a plan that fulfills certain structure.

Prefix-suffix structure

As a plan is essentially an infinite sequence of states in $\mathcal{T}_c$, it is not convenient to encode, analyze or manipulate both in theory and software implementation. Thus we consider the plan with the *prefix-suffix* structure:

$$\tau = \langle \tau_{\text{pre}}, \tau_{\text{suf}} \rangle = \tau_{\text{pre}} \left[\tau_{\text{suf}} \right]^{\omega} \tag{3.5}$$

where the prefix τ_{pre} is transversed only once and the suffix τ_{suf} is repeated infinitely. A plan with this prefix-suffix structure has a finite representation as (3.5). This structure is also called *lasso-shapeed* in [130] with the stem τ_{pre} and the loop τ_{suf}.

Definition 3.6. φ is called **feasible** if there exists an infinite path τ of $\mathcal{T}_c$ that satisfies φ. ▲

With the above preliminaries in hand, the problem formulation for the nominal scenario could be stated as follows:

Problem 3.1. *Given the control-driven wFTS $\mathcal{T}_c$ and an LTL formula φ over AP, (i) find a plan with the prefix-suffix structure in* (3.5)*; (ii) construct the hybrid control strategy based on* (3.3) *to execute the derived plan.* ▲

3.3 Hybrid controller synthesis

In this section, we describe in detail how to synthesize the discrete plan that solves the first part of Problem 3.1.

Büchi automaton

Given an LTL formula φ over AP, there exists a Nondeterministic Büchi automaton (NBA) over 2^{AP} corresponding to φ, denoted by $\mathcal{A}_\varphi$.

Definition 3.7. The NBA $\mathcal{A}_\varphi$ is defined by a five-tuple:

$$\mathcal{A}_\varphi = (Q, 2^{AP}, \delta, Q_0, \mathcal{F}), \tag{3.6}$$

where Q is a finite set of states; $Q_0 \subseteq Q$ is a set of initial states; 2^{AP} is the alphabet; $\delta : Q \times 2^{AP} \to 2^Q$ is a transition relation; $\mathcal{F} \subseteq Q$ is a set of accepting states. ▲

An infinite run of the NBA is an infinite sequence of states that starts from an initial state and follows the transition relation. Namely, $r = q_0 q_1 q_2 \cdots$, where $q_0 \in Q_0$ and $q_{k+1} \in \delta(q_k, S)$ for some $S \in 2^{AP}$, $k = 0, 1, \cdots$. Moreover, r is called *accepting* if $\texttt{Inf}(r) \cap \mathcal{F} \neq \varnothing$, where $\texttt{Inf}(r)$ is the set of states that appear in r infinitely often. We denote the successors of $q_m \in Q$ by $\texttt{Post}(q_m) = \{q_n \mid \exists S \in 2^{AP},\ q_n \in \delta(q_m, S)\}$.

Definition 3.8. Given an infinite word $\sigma = S_0 S_1 S_2 \cdots$ over 2^{AP}, its **resulting run** in $\mathcal{A}_\varphi$ is denoted by $r_\sigma = q_0 q_1 q_2 \cdots$, which satisfies: (i) $q_0 \in Q_0$; (ii) $q_{i+1} \in \delta(q_i, S_i)$, $\forall i = 1, 2, \cdots, \infty$. Similar statement holds for a finite word $\bar{\sigma} = S_0 S_1 S_2 \cdots S_{N+1}$. ▲

Note that since $\mathcal{A}_\varphi$ is nondeterministic, there may exist *multiple* resulting runs of the same world. Denote by $\mathcal{L}_\omega(\mathcal{A}_\varphi)$ the accepted language of $\mathcal{A}_\varphi$, which is the set of infinite words that result in an accepting run of $\mathcal{A}_\varphi$, i.e., $\mathcal{L}_w(\mathcal{A}_\varphi) = \{\sigma \in (2^{AP})^\omega \mid r_\sigma \text{ is an accepting run}\}$.

Lemma 3.1. $\mathcal{L}_w(\mathcal{A}_\varphi) = \texttt{Words}(\varphi)$

Proof. See proof of Theorem 5.37 in [11] ■

The translation process from an LTL formula to its corresponding NBA can be done in time and space $2^{\mathcal{O}(|\varphi|)}$ [11]. However, there are fast translation algorithms [41], which generates NBA with few states and transitions. Furthermore it is tedious to list all input alphabets for each transition, particularly given a large set of AP. Thus it is important to represent them in an compact and efficient manner. The translation algorithm from [41] generates a boolean expression for each transition, which accepts all alphabets that enable this transition (as shown in Figure 3.3). Binary decision diagrams (BDD) are well-known for their efficiency to represent and evaluate boolean functions [4] . As a result, an NBA can be encoded symbolically and efficiently. More details can be found in Section 8.

Example 3.3. The NBA that corresponds to $\varphi = (\Box \Diamond a_1) \wedge (\Box \Diamond a_2) \wedge (\Box \Diamond a_3) \wedge (\Box \neg a_4)$ is derived from [41] and shown in Figure 3.3. The transition from state q_1 to q_3 is given by $q_3 \in \delta(q_1, l)$ where the boolean expression $l = (a_2 \,\&\, \neg a_4)$ encodes four input alphabets $\{a_2\}$, $\{a_2, a_1\}$, $\{a_2, a_3\}$, $\{a_2, a_1, a_3\}$. ▲

3.3.1 Product Büchi automaton

The automaton-based model-checking algorithm can be found in [147] and Algorithm 11 of [11]. It is based on checking the emptiness of the product Büchi automaton. Since $\texttt{Words}(\varphi) = \mathcal{L}_w(\mathcal{A}_\varphi)$ and $\texttt{trace}(\tau) \in \texttt{Trace}(\mathcal{T}_c)$, the original problem is equivalent to finding the intersection $\texttt{Trace}(\mathcal{T}_c) \cap \mathcal{L}_w(\mathcal{A}_\varphi)$, which is actually the language of the product Büchi automaton $\mathcal{A}_p = \mathcal{T}_c \otimes \mathcal{A}_\varphi$, which accepts all runs that are valid for $\mathcal{T}_c$ and at the same time satisfy φ.

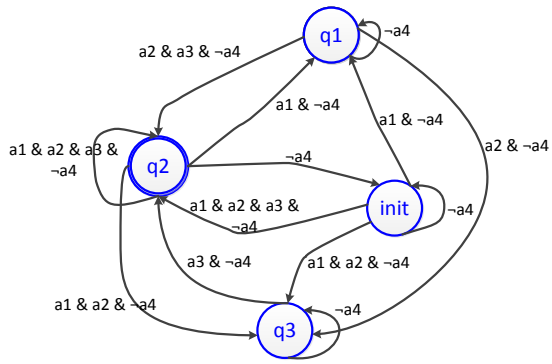

Figure 3.3: The NBA associated with $\varphi = (\Box \Diamond a_1) \wedge (\Box \Diamond a_2) \wedge (\Box \Diamond a_3) \wedge (\Box \neg a_4)$ (constructed by program from [41]), as explained in Example 3.3.

It is important to mention that unlike the model-checking algorithm for verification, we do not negate the task specification before generating the associated NBA and the product automaton. This is because we are interested in the "good" behavior of the system that satisfies the specification, not the "bad" behavior that satisfies the negated specification for the purpose of verification.

Definition 3.9. The weighted product Büchi automaton is defined by $\mathcal{A}_p = \mathcal{T}_c \otimes \mathcal{A}_\varphi = (Q', \delta', Q'_0, \mathcal{F}', W_p)$, where $Q' = \Pi \times Q = \{\langle \pi, q\rangle \in Q' \mid \forall \pi \in \Pi, \ \ \forall q \in Q\}$; $\delta' : Q' \to 2^{Q'}$. $\langle \pi_j, q_n\rangle \in \delta'(\langle \pi_i, q_m\rangle)$ iff $(\pi_i, \pi_j) \in \longrightarrow_c$ and $q_n \in \delta(q_m, L_c(\pi_i))$; $Q'_0 = \{\langle \pi, q\rangle \mid \pi \in \Pi_0, \ q_0 \in Q_0\}$, the set of initial states; $\mathcal{F}' = \{\langle \pi, q\rangle \mid \pi \in \Pi, q \in \mathcal{F}\}$, the set of accepting states; $W_p : Q' \times Q' \to \mathbb{R}^+$ is the weight function: $W_p(\langle \pi_i, q_m\rangle, \langle \pi_j, q_n\rangle) = W_c(\pi_i, \pi_j)$, where $\langle \pi_j, q_n\rangle \in \delta'(\langle \pi_i, q_m\rangle)$. ▲

Since $\mathcal{A}_p$ remains a Büchi automaton [11], its infinite run and its accepting condition can be defined similarly as $\mathcal{A}_\varphi$. An infinite run R is called accepting if it intersects with the accepting set $\mathcal{F}'$ infinitely often. The successors of q'_s are given by $\texttt{Post}(q'_s) = \{q'_g \mid q'_g \in \delta'(q'_s)\}$. Given a state $q' = \langle \pi, q\rangle \in Q'$, its projection on Π is denoted by $q'|_\Pi = \pi$ and its projection on Q is denoted by $q'|_Q = q$. Given an infinite run $R = q'_0 q'_1 q'_2 \cdots$ of $\mathcal{A}_p$, its projection on Π is denoted by $R|_\Pi = q'_0|_\Pi \, q'_1|_\Pi \, q'_2|_\Pi \cdots$ and its projection on Q is denoted by $R|_Q = q'_0|_Q \, q'_1|_Q \, q'_2|_Q \cdots$.

Lemma 3.2. *If there exists an infinite path τ of $\mathcal{T}_c$ such that $\tau \vDash \varphi$, then* ***at least*** *an accepting run of $\mathcal{A}_p$ exists.*

Proof. See the proof of Theorem 4.63 from [11]. ■

Lemma 3.3. *Given an accepting run R of $\mathcal{A}_p$, then $R|_\Pi \vDash \varphi$.*

Proof. By definition, there exists an accepting state $q'_f \in \mathcal{F}'$ appearing in R infinitely often. Thus $q'_f|_Q \in \mathcal{F}$ appears in $R|_Q$ infinitely often, yielding that $R|_Q$ is an accepting

run. It can be easily shown that $R|_Q$ is one of the resulting runs of the trace of $R|_\Pi$. Thus $\texttt{trace}(R|_\Pi) \in \mathcal{L}_\omega(\varphi) = \texttt{Words}(\varphi)$, which implies $R|_\Pi \vDash \varphi$ by Definition 3.5. ■

Cost of an accepting run

We intended to find an infinite path τ of $\mathcal{T}_c$ with the prefix-suffix structure in (3.5), such that $\tau \vDash \varphi$. The following lemma is important for the correctness.

Lemma 3.4. *If there exists an infinite path τ with the prefix-suffix structure and $\tau \vDash \varphi$, then **at least** one accepting run of $\mathcal{A}_p$ exists with the prefix-suffix structure.*

Proof. Since $\tau \vDash \varphi$, then $\sigma = \texttt{trace}(\tau) \in \texttt{Words}(\varphi)$. Furthermore as $\mathcal{L}_w(\mathcal{A}_\varphi) = \texttt{Words}(\varphi)$ by Lemma 3.1, $\sigma \in \mathcal{L}_w(\mathcal{A}_\varphi)$, meaning that the resulting run r_σ in $\mathcal{A}_\varphi$ by Definition 3.8 is an accepting run of $\mathcal{A}_\varphi$.

Without loss of generality, let $\tau = \pi_0\pi_1\cdots\pi_i\cdots$ and $r_\sigma = q_0q_1\ldots q_j\cdots$. Now we prove that $R = \langle\pi_0, q_0\rangle\langle\pi_1, q_1\rangle\cdots\langle\pi_i, q_j\rangle\cdots$ is an accepting run of $\mathcal{A}_p$. First of all, $\langle\pi_0, q_1\rangle \in Q_0'$ as $\pi_0 \in \Pi_0$ and $q_0 \in Q_0$. Secondly, $\langle\pi_{i+1}, q_{j+1}\rangle \in \delta'(\langle\pi_i, q_j\rangle)$ as $(\pi_i, \pi_{i+1}) \in \longrightarrow_c$ and $q_{i+1} \in \delta(q_i, L_c(\pi_i))$. At last, since r_σ is an accepting run of $\mathcal{A}_\varphi$, there exists an accepting state $q_f \in \mathcal{F}$ that appears in r_σ infinitely often. Correspondingly, there exists at least one $\langle\pi, q_f\rangle$ that appears in R infinitely often and $\langle\pi, q_f\rangle \in \mathcal{F}'$, where π may stand for one or several states in Π. Since $\mathcal{F}'$ is finite, there must be one accepting state $q_f' \in \mathcal{F}'$ that appears in R infinitely often.

Then an accepting run with the prefix-suffix structure can be constructed by using the segment from $\langle\pi_0, q_0\rangle$ to q_f' as prefix and the segment starting from q_f' and back to q_f' as the suffix, which completes the proof. ■

Thus we could focus on the accepting runs of $\mathcal{A}_p$ with the *prefix-suffix* structure:

$$\begin{aligned} R &= \langle R_{\text{pre}}, R_{\text{suf}}\rangle = q_0'\,q_1'\cdots q_f'\,\left[\,q_f'\,q_{f+1}'\cdots\cdots q_n'\,\right]^\omega \\ &= \langle\pi_0, q_0\rangle\cdots\langle\pi_{f-1}, q_{f-1}\rangle\,\left[\,\langle\pi_f, q_f\rangle\langle\pi_{f+1}, q_{f+1}\rangle\cdots\cdots\langle\pi_n, q_n\rangle\,\right]^\omega\,, \end{aligned} \tag{3.7}$$

where $q_0' = \langle\pi_0, q_0\rangle \in Q_0'$ and $q_f' = \langle\pi_f, q_f\rangle \in \mathcal{F}'$. Note that there are no correspondences among the subscripts. The prefix part $R_{\text{pre}} = (q_0'\,q_1'\cdots q_f')$ from an initial state q_0' to one accepting state q_f' that is executed only once while the suffix part $R_{\text{suf}} = (q_f'\,q_{f+1}'\cdots\cdots q_n')$ from q_f' back to itself that is repeated infinitely. Given the finite representation as (3.7), there is a finite set of transitions appearing in R:

$$\texttt{Edge}(R) = \{(q_i', q_{i+1}'),\ i = 0, 1\cdots, (n-1)\} \cup \{(q_n', q_f')\}. \tag{3.8}$$

The structure also allows us to define the total cost of R:

$$\begin{aligned} \texttt{Cost}(R, \mathcal{A}_p) &= \sum_{i=0}^{f-1} W_p(q_i', q_{i+1}') + \gamma \sum_{i=f}^{n-1} W_p(q_i', q_{i+1}') \\ &= \sum_{i=0}^{f-1} W_c(\pi_i, \pi_{i+1}) + \gamma \sum_{i=f}^{n-1} W_c(\pi_i, \pi_{i+1}) \end{aligned} \tag{3.9}$$

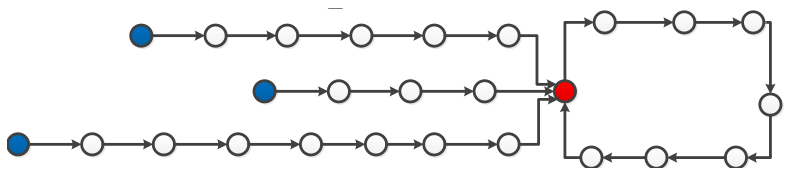

Figure 3.4: For every pair of $q'_0 \in Q'_0$ (in blue) and $q'_f \in \mathcal{F}'$ (in red), the shortest path from q'_0 to q'_f and the shortest cycle containing q'_f are computed.

where the first summation in (3.9) represents the accumulated weights of transitions along the prefix and the second the suffix; $\gamma \geq 0$ is the relative weighting on the cost of transient response (the prefix) and steady response (the suffix).

Problem 3.2. *Given the product automaton $\mathcal{A}_p$, find its accepting run with the prefix-suffix structure that* ***minimizes*** *the cost defined by* (3.9). ▲

We denote by R_{opt} the solution for the above problem, as the optimal accepting run. Its corresponding plan is given by $\tau_{\text{opt}} = R_{\text{opt}}|_{\Pi}$.

***Remark* 3.1.** τ_{opt} might not be the actual optimal plan with the prefix-suffix structure, whose cost is defined similarly as (3.9). As pointed out in [130], this optimality loss is due to the simplification during the translation process from LTL formulas to the corresponding NBA. However there are certain types of *tight* NBA such as [19] that preserves this quality. It means that the optimal plan can be found directly as the projection of the optimal run. But we have not found the software implementation for this translation algorithm yet. Nevertheless the trade-off between optimality and computational complexity remains as the tight NBA would certainly have far more states and edges as shown in the examples of [130]. ▲

3.3.2 Optimal run search

In this part we present firstly two methods to construct the product automaton and then the graph search algorithm to find the optimal accepting run.

Full construction

Since only static and fully-known workspace is considered, both $\mathcal{T}_c$ and φ are time-invariant. Thus $\mathcal{A}_p$ can be constructed fully by Definition 3.9. In particular, given a FTS state $\pi_i \in \Pi$ and a NBA state $q_m \in Q$, the composed state $\langle \pi_i, q_m \rangle$ is added to Q' if it is not in Q' already. Then all transitions originated from $\langle \pi_i, q_m \rangle$ can be found by the definition of δ', namely $\forall \pi_j \in \texttt{Post}(\pi_i)$ and $\forall q_m \in \texttt{Post}(q_n)$, $\langle \pi_j, q_n \rangle \in \texttt{Post}(\langle \pi_i, q_m \rangle)$ if $q_n \in \delta(q_m, L(\pi_i))$.

After constructing $\mathcal{A}_p$ fully, Algorithm 3.1 takes as input arguments $\mathcal{A}_p$ and a set of starting states $S' \subseteq Q'$, which is set to Q'_0 by default, while it generates the optimal accepting run. Algorithm 3.1 utilizes Dijkstra's algorithm [97] for computing the shortest path from a single source node to a set of target nodes within a weighted

Algorithm 3.1 Search for optimal run, `OptRun`().

Input: $\mathcal{A}_p$, $S' = Q'_0$ by default
Output: R_{opt}
1. If Q'_0 or $\mathcal{F}'$ is empty, construct Q'_0 or $\mathcal{F}'$ first.
2. For each initial state $q'_0 \in S'$, call `DijksTargets`(G, q'_0, $\mathcal{F}'$).
3. For each accepting state $q'_f \in \mathcal{F}'$, call `DijksCycle`(G, q'_f).
4. Find the pair of $(q'_{0,\star}, q'_{f,\star})$ that minimizes the summed cost defined by (3.9).
5. Optimal accepting run R_{opt}, prefix: the shortest path from $q'_{0,\star}$ to $q'_{f,\star}$; suffix: the shortest cycle from $q'_{f,\star}$ and back to itself.

graph. In particular, function `DijksTargets`($\mathcal{A}_p$, q'_S, Q'_T) computes shortest paths in $\mathcal{A}_p$ from source state $q'_S \in Q$ to every target state belonging to the set $Q'_T \in Q'$. Function `DijksCycle`($\mathcal{A}_p$, q'_S) is used to compute shortest cycle from the source state q'_S back to itself. As shown in Figure 3.4, for each pair of initial and accepting states (q'_0, q'_f) where $q'_0 \in Q'_0$ and $q'_f \in \mathcal{F}'$, the shortest path from q'_0 to q'_f is obtained from line 1 of Algorithm 3.1 where `DijksTargets`($\cdot$) is called while the shortest cycle containing q'_f is obtained from line 2 of Algorithm 3.1 where `DijksCycle`($\cdot$) is called. At last, the pair $(q'_{0,\star}, q'_{f,\star})$ that minimizes the total cost defined by (3.9) is chosen. Then the optimal accepting run is determined by setting its prefix as the shortest path from $q'_{0,\star}$ to $q'_{f,\star}$ and its suffix as shortest cycle containing $q'_{f,\star}$.

The algorithm to compute $\mathcal{A}_p$ has the complexity proportional to the sum of the number of states and transitions in $\mathcal{A}_p$. The worst-case complexity of Algorithm 3.1 is given by $\mathcal{O}(|\delta'| \cdot \log |Q'_0| \cdot (|Q'_0| + |\mathcal{F}'|)))$ as essentially the Dijkstra algorithm [89, 97] has to be called over $\mathcal{A}_p$ by the number of times equal to $|Q'_0| + |\mathcal{F}'|$.

On-the-fly construction

Beside constructing $\mathcal{A}_p$ once for all, we would construct $\mathcal{A}_p$ on-the-fly along with the graph search Algorithm 3.1 and even the revision Algorithm 4.4 introduced later. In other words, the states and transition relations of $\mathcal{A}_p$ are built "on demand". When the search algorithm visits any state $q'_s \in Q'$, it constructs the adjacency relation of q'_s by iterating through all successors of q'_s and returns the corresponding transition $q'_g \in \delta'(q'_s)$ along with its weight. Note that each state $q'_s \in Q'$ is marked by the label "visited" or "unvisited", to indicate if the transitions originated from q'_s have to be constructed. In particular, if q'_s is marked "visited", it means that they have been constructed before and can be returned directly; if q'_s is marked "unvisited" or π_i belongs to $\widehat{\Pi}$, they need to be constructed by Definition 3.9 ; $\widehat{\Pi}$ will be defined in (4.19), which can be treated as $\varnothing$ for now. Detailed implementation of the on-the-fly construction can be found in [48].

The markers "visited" and "unvisited" improve the computational efficiency as the adjacency relation of states that have been visited before can be returned directly, meaning that the results from previous planning iterations can be reused later; $\widehat{\Pi}$ provides a highly efficient way to maintain and update $\mathcal{A}_p$ in case of updates

Size n^2	$t_{\text{full}}\,(s)$	$t_{\text{syn}}\,(s)$	$t_{\text{fly}}\,(s)$	Size n^2	$t_{\text{full}}\,(s)$	$t_{\text{syn}}\,(s)$	$t_{\text{fly}}\,(s)$
25	1.11	0.03	0.82	3025	64.77	2.98	74.53
225	8.85	0.23	6.88	4225	91.65	3.96	101.34
625	14.04	0.56	11.09	5625	118.24	5.44	144.78
1225	24.96	1.12	23.24	7225	150.77	7.00	178.99
2025	42.40	1.87	38.13	9025	187.01	8.95	218.17

Table 3.1: Numerical results for Example 3.4.

in $\mathcal{T}_c$, as later described in Algorithm 4.4 in Section 4.3. Then Algorithm 3.1 can be easily modified such that it takes the adjacency relation of $\mathcal{A}_p$ as an input, instead of the fully-constructed $\mathcal{A}_p$. Functions `DijksTargets`$(\cdot)$ and `DijksCycle`$(\cdot)$ still work in the same way since the Dijkstra algorithm only needs the adjacency relation in the breadth-first structure and the associated weight. Even though the worst-case computational complexity of Algorithm 3.1 remains the same for these two different methods to construct $\mathcal{A}_p$, the on-the-fly construction is essential for the partially-known and dynamic workspace that will be discussed in Section 4.3, where $\mathcal{T}_c$ has to be updated frequently.

Example 3.4. This example shows the general complexity of synthesizing a discrete motion and task plan using the proposed scheme. The FTS consists of $n \times n$ uniform grids as states and the cost from one region to an adjacent region is set randomly and uniformly within $[0, 1]$. The task is to deliver two objects to two different destinations separately and then return to the base station. The formula can be written similarly as (3.13). The location of the objects and destinations are chosen randomly. We keep track of: (i) t_{full}, the time needed to fully construct $\mathcal{A}_p$ by Definition 3.9; (ii) t_{syn}, the time taken to find the optimal accepting run by Algorithm 3.1 over the fully-constructed $\mathcal{A}_p$; (iii) t_{fly}, the time taken to construct $\mathcal{A}_p$ on-the-fly along the optimal search by Algorithm 3.1. They are measured by CPU time in seconds on a desktop computer (3.06 GHz Duo CPU and 8GB of RAM).

The associated NBA has 75 states and 877 edges. Judging from Table 3.1, given a fixed transition system and this particular task, these two different approaches consume almost the same amount of time. The construction of $\mathcal{A}_p$ takes almost 95% of the total time while the graph search algorithm is relatively fast. ▲

3.3.3 Control structure

Assume the optimal run R_{opt} from Algorithm 3.1 has the following format:

$$\begin{aligned} R_{\text{opt}} &= \langle R_{\text{pre}},\, R_{\text{suf}} \rangle \\ &= R_{\text{pre},1} R_{\text{pre},2} \cdots R_{\text{pre},N_{\text{pre}}} \left[R_{\text{suf},1} R_{\text{suf},2} \cdots R_{\text{suf},N_{\text{suf}}} \right]^{\omega}, \end{aligned} \tag{3.10}$$

where $R_{\text{pre}} = R_{\text{pre},1} \cdots R_{\text{pre},N_{\text{pre}}}$ is the prefix; $R_{\text{suf}} = R_{\text{suf},1} \cdots R_{\text{suf},N_{\text{suf}}}$ is the suffix.

Algorithm 3.2 Plan execution by hybrid control, `HybCon( )`

Input: R_{opt}, $x(t)$
Output: u, R_{past}, τ_{past}, q'_{cur}
$q'_{\text{cur}} = q'_{\text{next}} = R_{\text{pre},1}$, $\pi_{\text{next}} = q'_{\text{next}}|_{\Pi}$, $R_{\text{past}} = [\ ]$, $\tau_{\text{past}} = [\ ]$
while `True` **do**
  if $x(t) \in \pi_{\text{next}}$ confirmed **then**
    $q'_{\text{cur}} = q'_{\text{next}}$, $\tau_{\text{past}} = \tau_{\text{past}} + \pi_{\text{next}}$, $R_{\text{past}} = R_{\text{past}} + q'_{\text{next}}$
    $q'_{\text{next}} = \texttt{NextGoal}(q'_{\text{cur}}, R_{\text{opt}})$, $\pi_{\text{cur}} = q'_{\text{cur}}|_{\Pi}$, $\pi_{\text{next}} = q'_{\text{next}}|_{\Pi}$
  $u(t) = \mathcal{U}(x(t), \pi_{\text{cur}}, \pi_{\text{next}})$

Thus the robot's status within the accepting run can be uniquely determined by the segment (prefix or suffix) and its index within that segment, which are denoted by seg and k. To generate the infinite plan using R_{pre} and R_{suf}, function $\texttt{NextGoal}(\cdot)$ takes the current product state $q'_{\text{cur}} = R_{seg,k}$ and R_{opt} as inputs and generates the next goal product state. Simply speaking, it firstly follows the prefix until the end of prefix. Then it switches to the suffix and follows it until the end. After that it restarts from the beginning of the suffix and repeats the same process. In this way, the suffix is executed infinitely many times.

Algorithm 3.2 executes the derived optimal run R_{opt} off-line. Initially the agent starts from $q'_{\text{pre},1}$; q'_{cur} and q'_{pre} are the current and next product state in R_{opt}; π_{cur} and π_{next} indicate the robot's current region and the next goal region. π_{next} is initialized as $\tau_{\text{pre},1}$; R_{past} is used to store the sequence of product states that has been reached in R_{opt}; τ_{past} is used to store the sequence of regions the robot has visited; Once a confirmation is acquired that π_{next} is reached, q'_{next} is added to R_{past} and correspondingly π_{next} is added to τ_{past}. Then q'_{next} is set to the next goal state given q'_{cur}. As a result, the controller $\mathcal{U}(x(t), \pi_{\text{cur}}, \pi_{\text{next}})$ is activated to drive the agent from π_{cur} to π_{next}. Algorithm 3.2 can be running for infinitely long time as the the suffix segment is repeated infinitely many times. Note that the condition $x(t) \in \pi_{\text{next}}$ holds when the robot belongs to its current goal region.

3.4 Case study

In this case study, we validate the proposed framework above by simulation results, while experiment results are presented in Chapter 8.

Workspace abstraction

The workspace is a testbed with size $2.4m \times 2.1m$ representing the office environment shown in Figure 3.1, consisting of three rooms on each side and one corridor in the middle. The corridor is partitioned into three smaller segments. Thus this workspace has nine regions in total "$r1, \cdots, r6, c1, c2, c3$", represented by propositions "r1,$\cdots$, r6, c1, c2, c3". There are one red ball, one green ball and

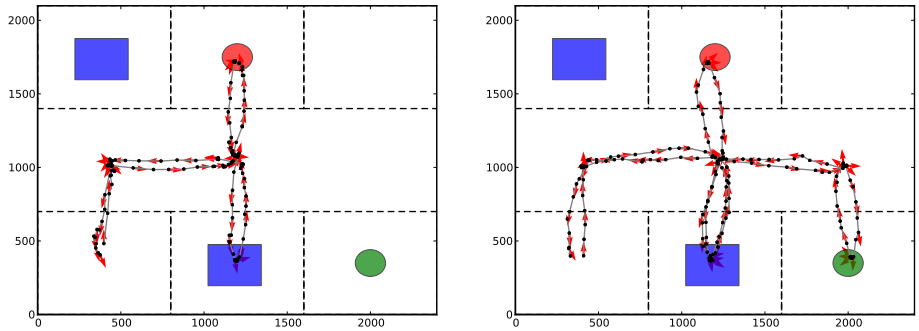

Figure 3.5: Left: the trajectory that fulfills φ_1. Right: the trajectory that fulfills φ_2.

two baskets in different rooms, represented by propositions "rball, gball, basket". The rectangular regions are encoded by the center point, width and height. The horizontally adjacent rooms are separated by vertical walls. The cost of moving from one region to another is estimated by the Euclidean distance between their centers. The region name and its labeling function are given by (from bottom to up, left to right): $(r1, \{\mathsf{r1}\})$, $(r2, \{\mathsf{r2}, \mathsf{basket}\})$, $(r3, \{\mathsf{r3}, \mathsf{gball}\})$, $(c1, \{\mathsf{c1}\})$, $(c2, \{\mathsf{c2}\})$, $(c3, \{\mathsf{c3}\})$, $(r4, \{\mathsf{r4}, \mathsf{basket}\})$, $(r5, \{\mathsf{r5}, \mathsf{rball}\})$, $(r6, \{\mathsf{r6}\})$. The robot we deployed is a NAO robot [5], which is an autonomous, programmable humanoid robot. Its state within the workspace is given by (x_r, y_r, θ_r), where x_r and y_r are the coordinate and $\theta_r \in [-\pi, \pi]$ is its orientation with respect to the x-axis. It has three basic control modules as "move_x$(\cdot)$", "move_y$(\cdot)$" and "turn$(\cdot)$". Namely, it can move forward in its local x-axis by the given speed, move sideways in its local y-axis by the given speed and it can turn itself by the given angular speed. Since they are not free of actuation noises and disturbances, we design the following turn-and-forward feedback controller:

$$u = \begin{cases} \mathsf{move_x}\big(\kappa_1 \cdot \|(x_g - x_r,\, y_g - y_r)\|\big), & \text{if } |\theta_{\text{dif}}| > \bar{\theta} \\ \begin{cases} \mathsf{turn}\big(\kappa_2 \cdot \theta_{\text{dif}}\big), & \text{if } \bar{\theta} < |\theta_{\text{dif}}| \le \pi \\ \mathsf{turn}\big(-\kappa_2 \cdot \mathtt{sign}(\theta_{\text{dif}}) \cdot (2\pi - \mathtt{abs}(\theta_{\text{dif}}))\big), & \text{if } |\theta_{\text{dif}}| > \pi \end{cases} \end{cases} \tag{3.11}$$

where (x_g, y_g) is the goal position; $\|x_g - x_r, y_g - y_r\|$ is the relative distance; $\theta_{\text{ref}} = \arctan(y_g - y_r, x_g - x_r)$ is the relative angle between the robot's position and the goal position; $\theta_{\text{dif}} = \theta_{\text{ref}} - \theta_r$ is the difference between robot's orientation and the desired orientation; $\bar{\theta} < \pi$ is a design parameter deciding when the robot should move forward; $\kappa_1, \kappa_2 > 0$ are design parameters as the proportional gain. Thus the robot would keep turning until it is facing the the goal position. The time interval to update u is also an important tunning parameter. It is verified by the simulation and experimental results that the proposed controller is effective. Then the generic controller $\mathcal{U}(x(t), \pi_i, \pi_j)$ in (3.3) can be obtained by setting the way point in (3.11)

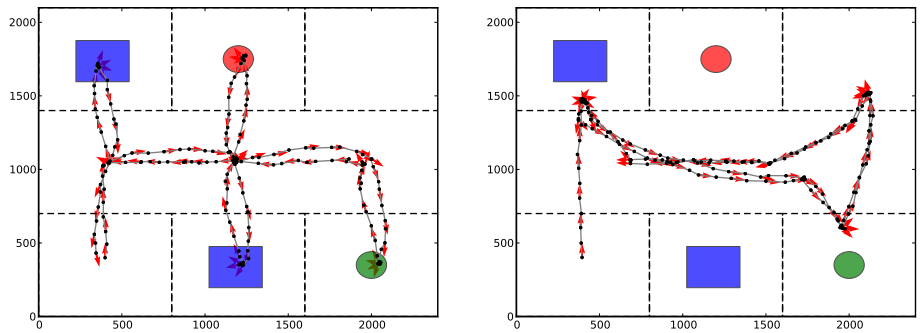

Figure 3.6: Left: the trajectory that fulfills φ_3. Right: the trajectory that fulfills φ_4.

as the center of the next goal region, which guarantees the robot's transition from one region to an adjacent one, except when there are walls in between. Based on this transition relation, we could construct $\mathcal{T}_c$ that consists of 9 regions and 16 edges. In the simulation, we add Gaussian noises to the actuation signal generated by (3.11).

Simulation results

We illustrate the effectiveness of the proposed framework by considering four different task specifications.

Case I: the robot has to pick up the red ball, drop it to the one of the baskets and then stay at room one. It is specified as the LTL formula: $\varphi_1 = \Diamond(\,\mathsf{rball} \wedge \Diamond\,\mathsf{basket}) \wedge \Diamond\,\Box\,\mathsf{r1}$, which can be interpreted as "eventually pick up the red ball. Once it is done, move to one basket and drop it. At last come back to room one and stay there". It took $0.03s$ for Algorithm 3.1 to find the optimal plan: "$r1\,c1\,c2\,r5\,c2\,r2\,c2\,c1\,r1\,(r1)^{\omega}$", with the prefix cost 581 and suffix cost 1. Then the optimal plan is constructed and executed off-line by Algorithm 3.2 and the final trajectory is shown in Figure 3.5.

Case II: same delivery task as in Case I, but now it has to deliver two objects and is not allowed for the robot to carry two objects simultaneously. The task is specified by the LTL formula:

$$\begin{aligned}\varphi_2 = {} & \Diamond\,(\,\mathsf{rball} \wedge \Diamond\,\mathsf{basket}) \wedge \Diamond(\,\mathsf{gball} \wedge \Diamond\,\mathsf{basket}) \wedge \Diamond\,\Box\,\mathsf{r1} \\ & \wedge\, \Box(\,\mathsf{rball} \Rightarrow \bigcirc(\,\neg\mathsf{gball}\ \mathsf{U}\ \mathsf{basket})) \\ & \wedge\, \Box(\,\mathsf{gball} \Rightarrow \bigcirc(\,\neg\mathsf{rball}\ \mathsf{U}\ \mathsf{basket})),\end{aligned} \tag{3.12}$$

where we add in the constraints that another ball can be picked up only after the robot has dropped the ball in hand. It took $0.86s$ for Algorithm 3.1 to find the optimal plan: "$r1\,c1\,c2\,c3\,r3\,c3\,c2\,r2\,c2\,r5\,c2\,r2\,c2\,c1\ (r1)^{\omega}$", with the prefix cost 1021 and suffix cost 1. The final trajectory is shown in Figure 3.5. It can be seen that the robot picks up the green ball first, drop it in the basket in $r2$, picks up the green ball, and drop it in the basket in $r2$, which fulfills the imposed task.

Case III: same delivery task as in Case II, but now we require that the red ball has to be delivered to the basket in $r2$ while the green ball has to the basket in $r4$. Now the task can be specified by:

$$\begin{aligned}\varphi_3 = \ & \Diamond\,(\,\mathsf{rball} \wedge \Diamond\,(\,\mathsf{basket} \wedge \mathsf{r2})) \ \wedge\ \Diamond(\,\mathsf{gball} \wedge \Diamond\,(\,\mathsf{basket} \wedge \mathsf{r4})) \\ & \wedge\ \Box(\,\mathsf{rball} \Rightarrow \bigcirc(\,\neg\mathsf{gball}\ \mathsf{U}\ \mathsf{basket})) \\ & \wedge\ \Box(\,\mathsf{gball} \Rightarrow \bigcirc(\,\neg\mathsf{rball}\ \mathsf{U}\ \mathsf{basket}))\ \wedge\ \Diamond\,\Box\ \mathsf{r1},\end{aligned} \tag{3.13}$$

where the location for each basket is specified. It took $1.39s$ for Algorithm 3.1 to find the optimal plan: "$r1\,c1\,c2\,r5\,c2\,r2\,c2\,c3\,r3\,c3\,c2\,c1\,r4\,c1\ (r1)^{\omega}$", with the prefix cost 1021 and suffix cost 1. The final trajectory is shown in Figure 3.6. It shows that the robot picks up the red ball first, drop it in the basket in $r2$, picks up the green ball, and drop it in the basket in $r4$.

Case IV: given a surveillance task, the robot needs to inspect rooms $r3$, $r4$ and $r6$ infinitely often. It can be written as the LTL formula: $\varphi_4 = (\Box\Diamond\ \mathsf{r3}) \wedge (\Box\Diamond\ \mathsf{r4}) \wedge (\Box\Diamond\ \mathsf{r6})$. It took $0.01s$ for Algorithm 3.1 to find the optimal plan: "$r1\,c1\,r4\,c1\,c2\,c3\,r3\,c3\,(r6\,c3\,c2\,c1\,r4\,c1\,c2\,c3\,r3\,c3)^{\omega}$", with the prefix cost 1021 and suffix cost 1. The final trajectory is shown in Figure 3.6. It can be seen that the robot patrols these three rooms as required.

3.5 Summary

In this chapter, we presented the framework to synthesize the hybrid control strategy for a mobile robot to fulfill a high-level temporal task. We started from constructing the finite abstraction of the robot's motion within the workspace. Then we proposed an automated scheme to synthesize the discrete motion and task plan. At last, a hybrid control strategy was designed for the robot to execute this discrete plan such that its resulting trajectory satisfies the given task.